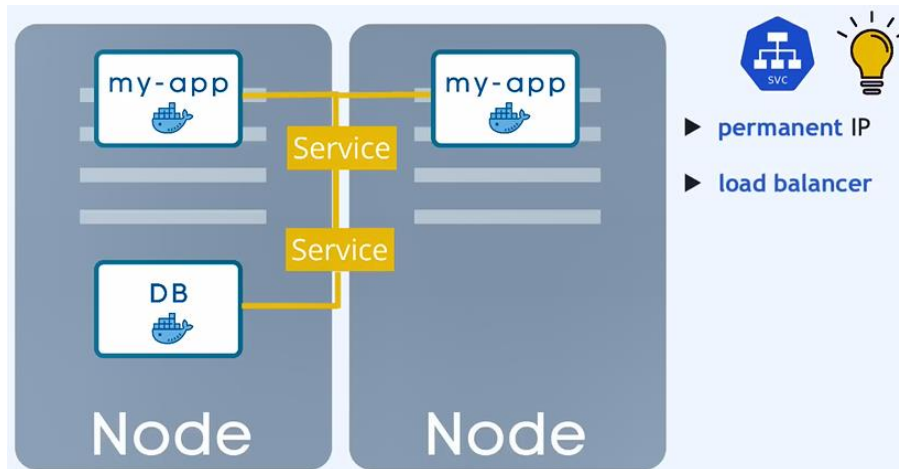
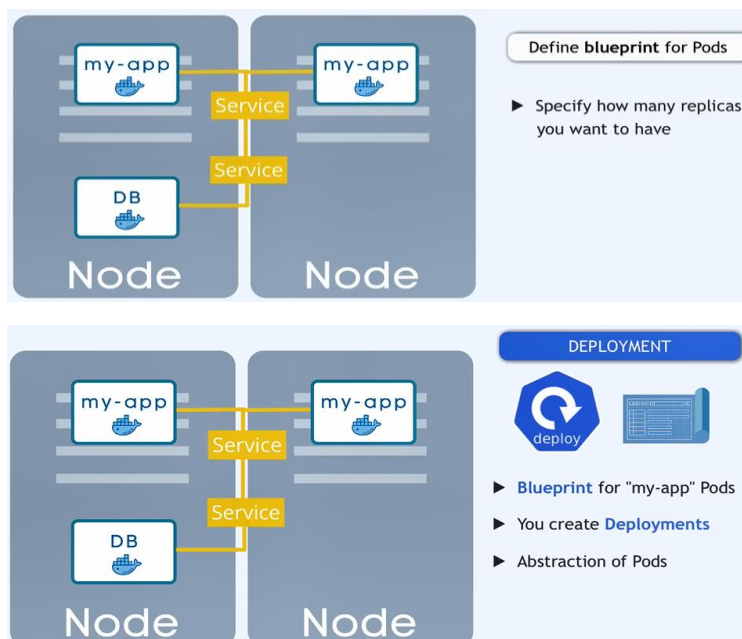


The *advantage of using distributed systems* or *microservices* with *containers* is that we have many replicas of our app, so if one of them is crashed or down, the others *will still handle the requests of users*

The other advantage of *service* that it *contains load balancer*



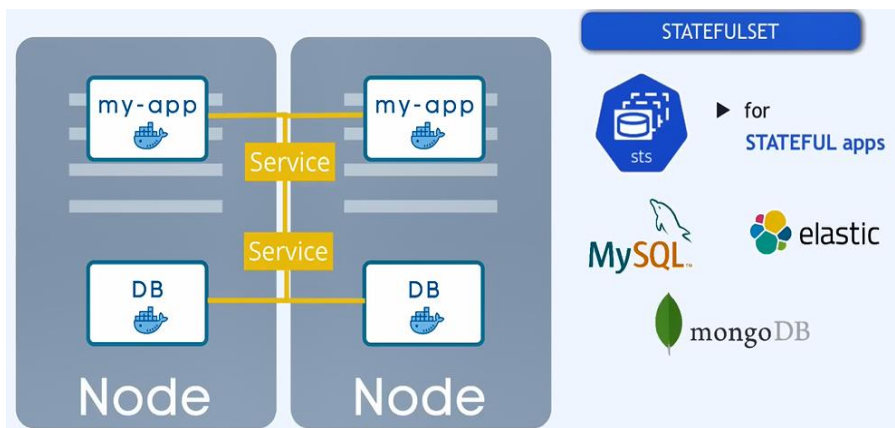
This blueprint called deployment, they are abstractions of Pod



We can't replicate the DB using deployments, because it has its state (data with volume):

! DB can't be replicated via Deployment!

The solution for previous is:



But the deployment for DB in same cases may not be easy



If one of the replicas die, then:

Controller Manager checks:
desired state == actual state ?

Each Configuration File has 3 Parts

Deployment

```
! nginx-deployment.yaml x
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: nginx-deployment 1
5    labels: ...
6  spec:
7    replicas: 2 2
8    selector: ...
9    template: ...
12
22
```

1) metadata

2) specification

Service

```
! nginx-service.yaml x
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: nginx-service 1
5  spec:
6    selector: ... 2
7    ports: ...
12
```

```
! nginx-deployment.yaml x
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: nginx-deployment
5    labels: ...
6  spec:
7    replicas: 2
8    selector: ...
9    template: ...
12
22
```

```
! nginx-service.yaml x
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: nginx-service
5  spec:
6    selector: ...
7    ports: ...
12
```

► Attributes of "spec" are **specific** to the **kind**

Each Configuration File has 3 Parts

1) metadata

2) specification

3) status

► **Automatically generated** and **added** by Kubernetes!

For status, kubernetes will always check the desired && actual status:

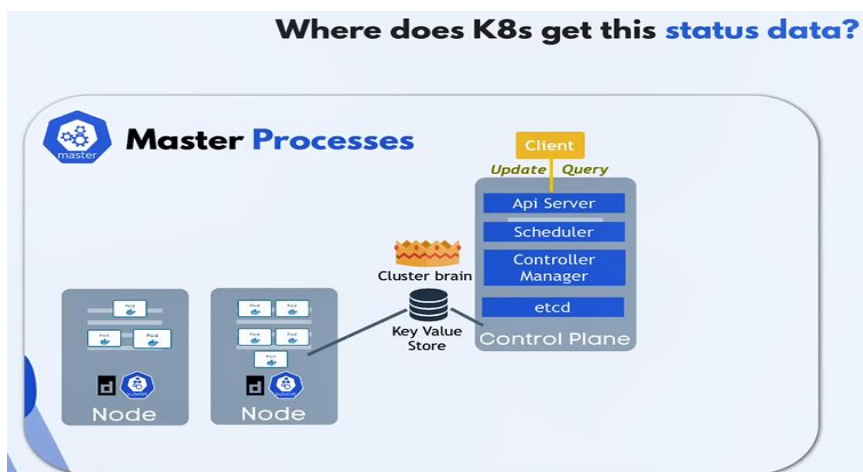
- Then we have choice of the desired = actual then it will still works
- Else, kubernetes will try to solve the problem.

```
! nginx-deployment.yaml x
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: nginx-deployment
5    labels: ...
6  spec:
7    replicas: 2
8    selector: ...
9    template: ...
10
```

3rd part: status

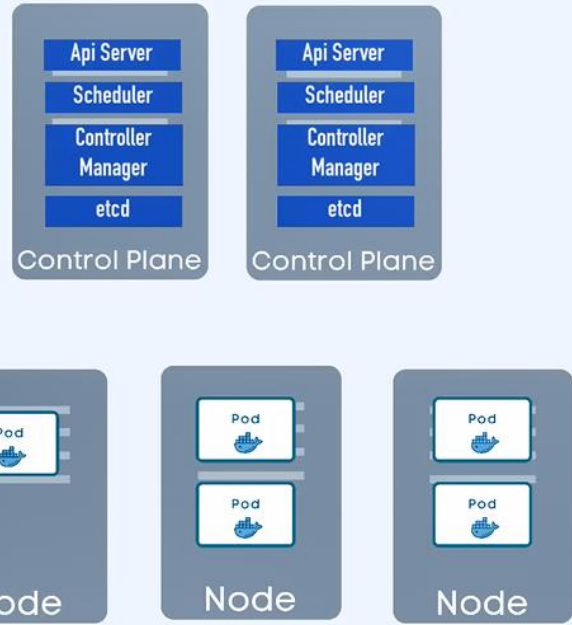
Desired? Actual?

Kubernetes save the important data (like status) in **etcd** (**key/value store**)

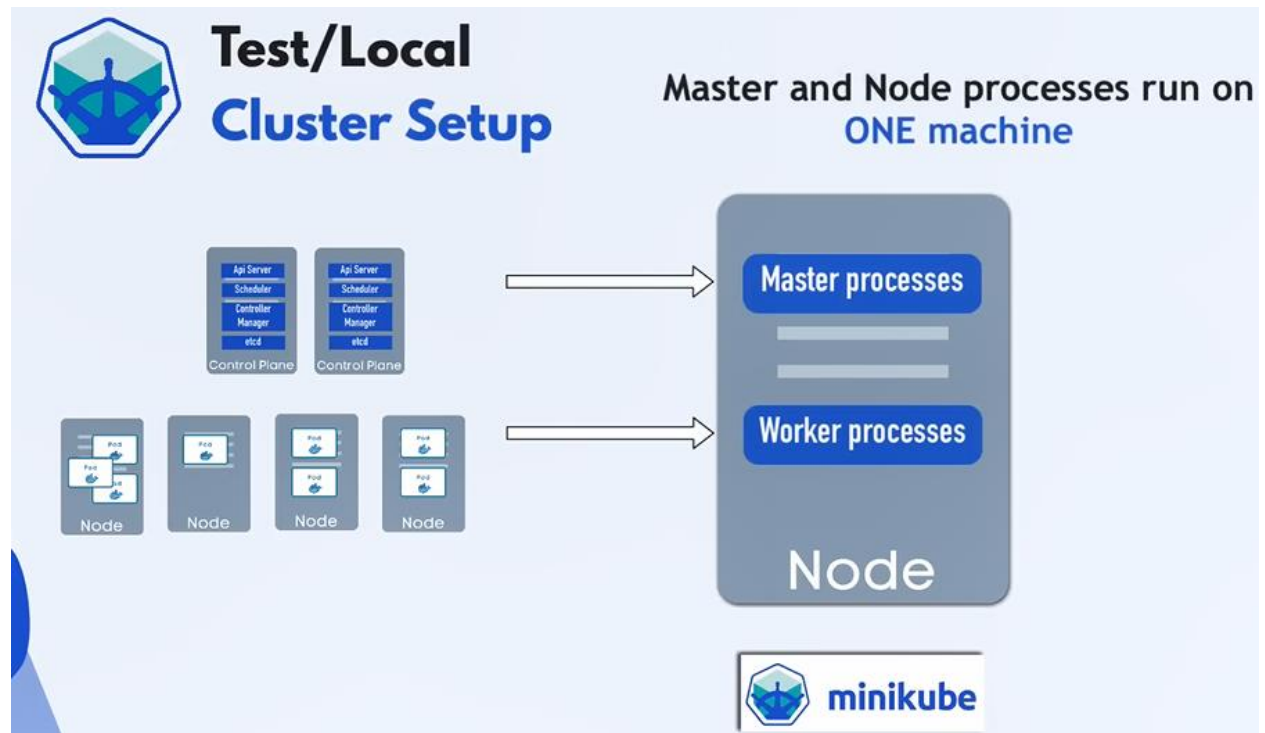


Production Cluster Setup

- Multiple Master and Worker nodes
- Separate virtual or physical machines

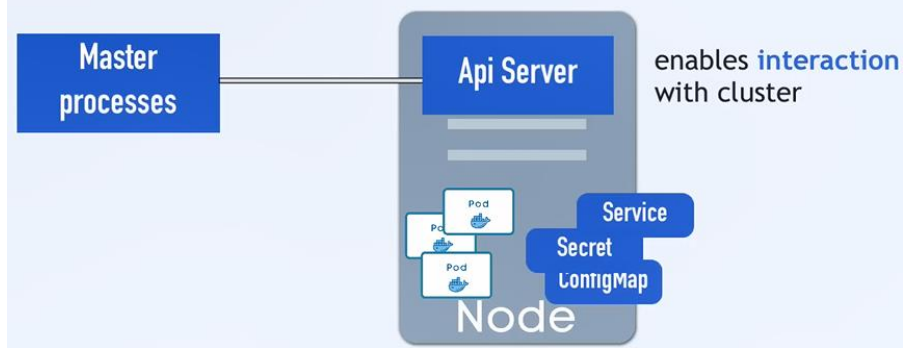


In test environment the *master* & *worker processes* are running *on the same node*:

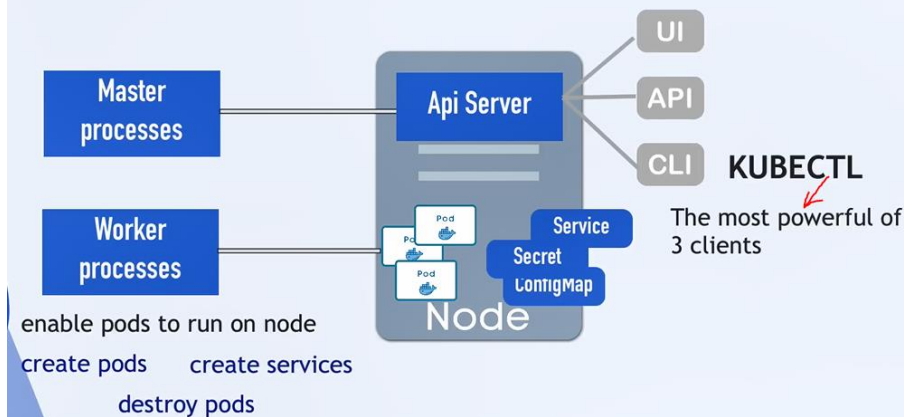


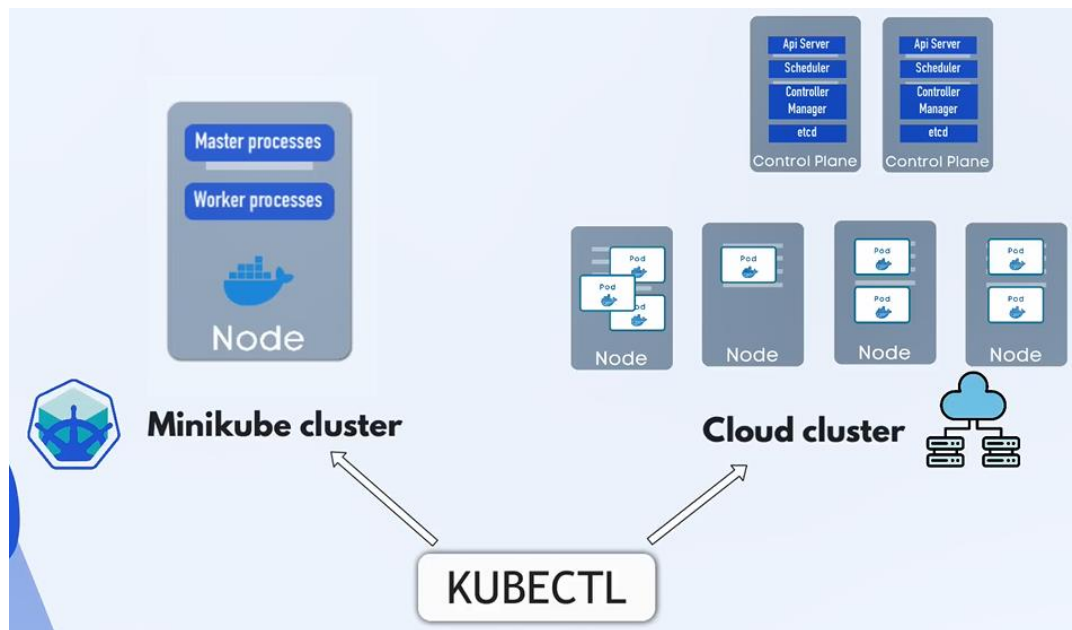
The main entry for kubernetes cluster is Api Server:

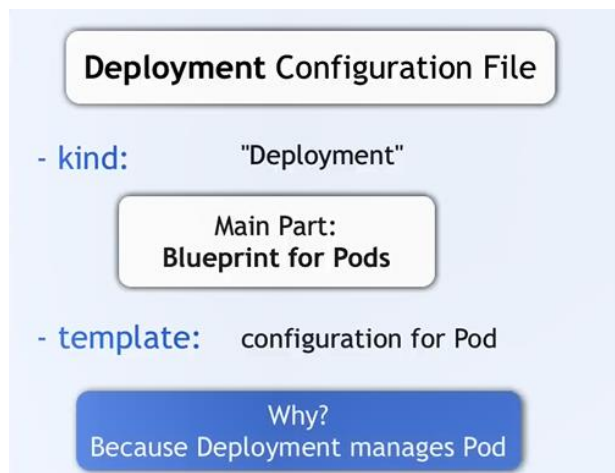
What is kubectl?



What is kubectl?



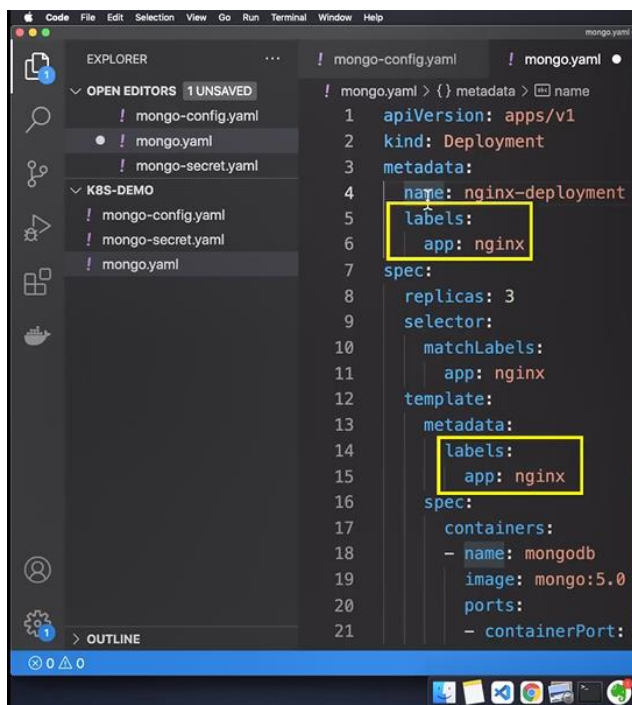






For spec in template we may have multiple containers per pod, but we usually have one main app per pod:

- **template:** configuration for Pod
has its own "metadata" and
"spec" section
- **containers:** which image?
which port?



```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx-deployment
5   labels:
6     app: nginx
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16     spec:
17       containers:
18       - name: mongod
19         image: mongo:5.0
20         ports:
21         - containerPort: 27017
```

- Label**
- ▶ You can give any K8s component a **label**
 - ▶ Labels are **key/value** pairs that are attached to K8s resources
 - ▶ Identifier, which should be meaningful and relevant to users

- Examples**
- "release" : "stable" "release" : "canary"
 - "env" : "dev" "env" : "production"
 - "tier" : "frontend" "tier" : "backend"

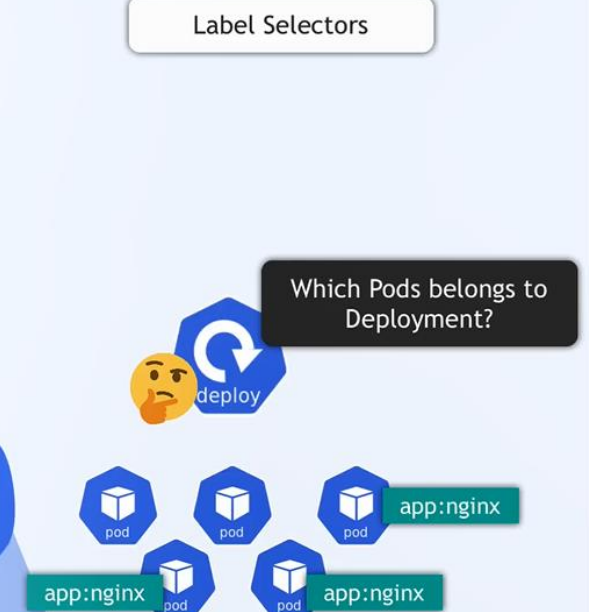
```
3 metadata:
4   name: nginx-deployment
5   labels:
6     app: nginx
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16   spec:
17     containers:
18     - name: mongodb
19       image: mongo:5.0
20       ports:
21       - containerPort: 27017
```

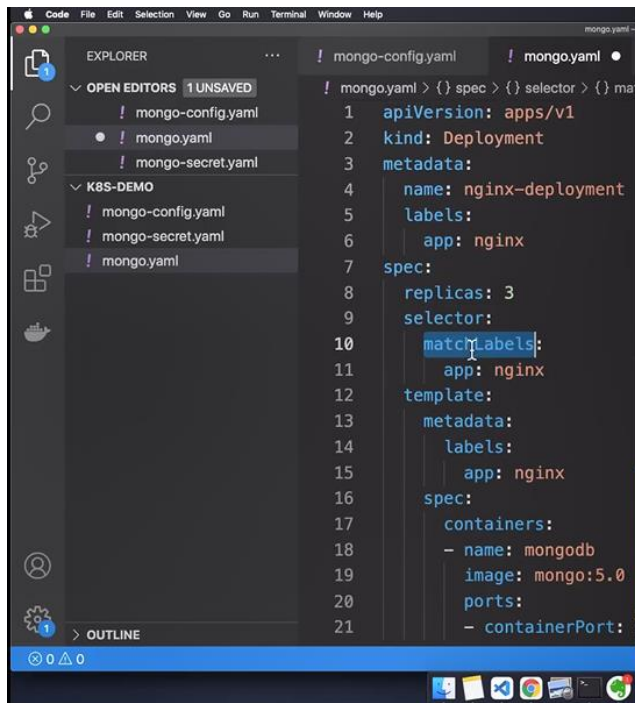
- ### Label
- ▶ Labels do not provide uniqueness.
E.g. all Pod replicas will have the same label

- ### 1 Use Case
- ▶ Connecting Deployment to all Pod replicas



```
3 metadata:
4   name: nginx-deployment
5   labels:
6     app: nginx
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16   spec:
17     containers:
18     - name: mongodb
19       image: mongo:5.0
20       ports:
21       - containerPort: 27017
```





The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'K8S-DEMO' with files 'mongo-config.yaml', 'mongo-secret.yaml', and 'mongo.yaml'. The main editor displays the 'mongo.yaml' file, which contains a Kubernetes Deployment manifest for nginx. The manifest includes fields for apiVersion, kind, metadata (name, labels), and spec (replicas, selector, template). The selector is configured to match pods with the label 'app: nginx'. The template specifies a container named 'mongodb' using the 'mongo:5.0' image.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx-deployment
5   labels:
6     app: nginx
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16     spec:
17       containers:
18       - name: mongodb
19         image: mongo:5.0
20       ports:
21       - containerPort: 27017
```

Label Selectors

- Identify a set of resources
- Match all Pods with label "app:nginx"

